PCPP IT University of Copenhagen, F2026

Exercises week 8

Last update 2026-10-19

These exercises aim to give you practical experience with performance measurements of:

- Java constructs such as threads and volatile;
- algorithms exploiting multiple threads to improve performance of e.g. sorting.

Do this first

The exercises build on the lecture, the lecture note *Microbenchmarks in Java, C# and C* and the accompanying example code. Carefully study the hints and warnings in Section 7 of that note before you measure anything, and:

NEVER measure from inside an IDE or when in Debug mode.

All the Java code in the *Microbenchmarks* note, the lecture, and these exercises can be found on the Github page for week 8: (https://github.itu.dk/PCPP/PCPP2026-Public/tree/main/week08). The code for the exercises is in:

`.../week08/code-exercises/week08exercises/....`

You will run some of the measurements discussed in the *Microbenchmarks* note yourself, and save results to text files. Use the `SystemInfo` method to record basic system identification, and supplement with whatever other information you can find about your execution platform:

- on Linux you may use `cat /proc/cpuinfo`;
- on MacOS you may use Apple > About this Mac, and even continue with > More Info > System Report;
- on Windows, look at System Information.

Exercise 8.1 In this exercise you must measure, on your own hardware, the time it takes to execute the built-in mathematical functions `pow`, `exp` and so on. This exercise involves no threads or parallel programming.

Mandatory

1. Uncomment the call to method `MathFunctionBenchmarks` in the `main` method of `Benchmark.java`, comment out all other calls, compile and run it.

   Report the resulting numbers. Are they plausible? Do you get similar numbers running the program again?

2. Using `Mark7`, measure the time to compute the square root with function `Math.sqrt`. Measure the time to generate a random number with `Math.random`.

   Report the resulting numbers. Are they plausible?

3. Measure the time to compute the square root of a random number, as in `Math.sqrt(Math.random())`.

   Report the resulting numbers. Are the numbers consistent with each other? If not, what could explain this?

Exercise 8.2 In this exercise you must measure, on your own hardware, the time it takes to create, start and join platform threads as well as virtual threads, using the same code `TestTimeThreads.java` as in the lecture.

Mandatory

1. First compile and run the code in `TestTimeThreads.java` as is. It uses `Mark6`, where the intermediate output gives a feeling for the variation and robustness of the results. Do not hand in the results but discuss any strangenesses, such as large variation in the time measurements for each case.

2. Now change all the measurements to use `Mark7`, which reports only the final result. Record the results in a text file along with appropriate system identification.

   Include the results in your hand-in, and reflect and comment on them: Are they plausible? Any surprises? Mention any cases where they deviate significantly from those shown in the lecture.

3. Use the same reasoning as in the lecture to estimate the cost of creating, starting and joining a platform thread on your own computer, subtracting the "thread work" part. Likewise estimate the cost of creating, starting and joining a virtual thread. Include your result and reflections in the text answers for this exercise.

Exercise 8.3 In this exercise you measure the performance of some sorting algorithms, including a simple parallel version of quicksort. Code is in `TestTimeSort.java`.

Mandatory

1. First measure the time to execute a quicksort algorithm on an array of 100,000 integers, like this:

```
final int[] intArray = Quicksort.fillIntArray(100_000);
Mark8Setup('u', "quicksort", "",
           () -> Quicksort.shuffle(intArray),
           i -> { Quicksort.quicksort(intArray); return intArray[37];
               });
```

   Report the measured execution time, which is in microseconds due to the `'u'` argument.

2. Still using `Mark8Setup`, measure also the time it takes to (shuffle and) sort the same array using Java's sequential `Arrays.sort` method.

   Report the measured execution time. Is it plausible? Is Java's library method faster than our own quicksort?

3. Do the same with Java's `Arrays.parallelSort` method.

   Report the measured execution time. Is it plausible? Is Java's parallel sort faster than the sequential one?

4. Now make a parallel version of our own quicksort. The implementation is in nested class Quicksort in the `TestTimeSort.java` file, where the actual work is done by method `qsort`, here shown in outline:

```
static void qsort(int[] arr, int a, int b) { // sort arr[a..b]
  if (a < b) {
    int i = a, j = b;
    int x = arr[(i+j) / 2];
    do {
      // not shown: code to partition into arr[a..j] and arr[i..b]
    } while (i <= j);
    qsort(arr, a, j);
    qsort(arr, i, b);
  }
}
```

   The interesting parts are the two recursive calls to `qsort` after the do-while loop. Those two calls operate on different parts of the array and can be executed in parallel.

   Rewrite that part of the code so that it creates and starts a new thread that executes `qsort(arr, a, j)` in parallel with `qsort(arr, i, b)`. The latter call should be left as is, so it executes on the already existing thread.

   Hint: Use virtual threads as in `Thread.ofVirtual().start(() -> ...)`. If you use platform threads, your program will likely crash with a huge number of OutOfMemoryError messages when it runs out of underlying operating system threads.

   Hint: Remember to `join` the new thread **after** the normal recursive call, so that it has terminated before the outer `qsort` call returns. The order should be: (a) create and start a new thread `t` for the first recursive call; (b) execute the second recursive call; and (c) join the `t` thread. This requires around five lines of code.

   Report the measured execution time. Is your parallel quicksort faster than the sequential one?

5. In the previous question you may be disappointed to find that the parallel quicksort is slower than the sequential one. This is likely because it unconditionally creates a new thread even to sort very short array

segments `arr[a..j]`. Now improve the code to create a new thread only when `j-a > 50` and otherwise just call `qsort(arr, a, j)` before `qsort(arr, i, b)` as in the original code.

Report the measured execution time. Is your new parallel quicksort faster than the sequential one?

Challenging

6. Define a method `static void runParallel(Runnable task1, Runnable task2)` which runs `task1` on a new virtual thread in parallel with `task2`, encapsulating the machinery you made above. Rewrite your parallel quicksort to use it.

Exercise 8.4 In this exercise you measure the performance impact, if any, of declaring an object field volatile. File `TestTimeVolatile.java` declares a class LongHolder that has both a non-volatile field `counter` and a volatile field `counterVolatile`, and methods that manipulate them:

```
class LongHolder {
  private long counter;
  private volatile long counterVolatile;

  public long value() { return counter; }
  public void add(long delta) { counter += delta; }
  private void callAdd(long i) {
    for (int j=0; j<1_000_000; j++)
      add(i+j);
  }

  public long valueVolatile() { return counterVolatile; }
  public void addVolatile(long delta) { counterVolatile += delta; }
  private void callAddVolatile(long i) {
    for (int j=0; j<1_000_000; j++)
      addVolatile(i+j);
  }
}
```

Mandatory

1. Use `Mark7` from `Benchmark.java` to measure the time to create an instance `lh` of LongHolder, call `callAdd(i)` on it, and return the LongHolder's `value`. Do **not** create a new thread to do this, but run the measurement on the main thread.

2. In the same way, measure the time to create an instance `lh` of LongHolder, call `callAddVolatile(i)` on it, and return the LongHolder's `valueVolatile`.

   Include the results in your hand-in and comment on them: Are they plausible? Any surprises? Is there a performance overhead when a field is declared volatile, and if so, how big is it?

   Challenging

3. Above, all updates to the fields were done on the same thread, which may underestimate the actual overhead caused by declaring the field volatile. Here are two plausible hypotheses: (1) When two distinct threads execute `callAddVolatile` at the same time, then those calls will be significantly slower because of memory overhead caused by Java's visibility requirements for volatile. (2) On the other hand, `callAdd` does not become slower, because there are no constraints on the visibility of non-volatile fields.

   Your task is to measure the time to create and run a virtual thread that calls `callAdd` in parallel with the main thread calling `callAdd`. Separately, measure the time to create and run a virtual thread that calls `callAddVolatile` in parallel with the main thread calling `callAddVolatile`.

   Include the results in your hand-in and comment on them: Are they plausible? Any surprises? Do the results confirm the hypotheses?